

O Problema da Mochila Bidimensional 0-1: Uma Abordagem Heurística

Arthur A. R. da Silva¹

¹Departamento de Informática – Universidade Tecnológica Federal do Paraná (UTFPR)
Via do Conhecimento, km 01 – 85.503-390 – Pato Branco – PR

arthursilva@utfpr.edu.br

Abstract. *This article addresses the Two-Dimensional 0-1 Knapsack Problem (2D-0/1 KP), a combinatorial optimization challenge classified as NP-hard, which is essential in logistics and engineering scenarios where items must simultaneously respect weight and volume constraints. The main objective was to implement and compare heuristic and metaheuristic approaches developed in the C programming language, aiming to overcome the infeasibility of exact methods such as Dynamic Programming (DP), whose memory cost of $O(W \times V)$ becomes prohibitive for large-scale instances. Variants of the greedy heuristic (based on linear efficiency and pseudo-utility) were evaluated, followed by refinement via Local Search (Hill Climbing) and the GRASP (Greedy Randomized Adaptive Search Procedure) metaheuristic. The experiments conducted on eight test instances (up to 1000 items) revealed that Local Search is highly dependent on the quality of the initial solution, reaching only 47.75% of the optimum in random starts in the largest-scale instance. In contrast, GRASP consolidated itself as the most robust strategy, maintaining consistent profits above 99% of the absolute optimum, with a massive speedup of up to 1.1 million times compared to DP. It is concluded that the hybridization of the greedy method with GRASP represents the ideal solution for real-time decision systems, balancing maximum profitability and operational speed.*

Resumo. *Este artigo aborda o Problema da Mochila Bidimensional 0-1 (2D-0/1 KP), um desafio de otimização combinatória classificado como NP-difícil, essencial em cenários de logística e engenharia onde itens devem respeitar limites simultâneos de peso e volume. O objetivo principal foi implementar e comparar abordagens heurísticas e metaheurísticas desenvolvidas em linguagem C, visando superar a inviabilidade de métodos exatos como a Programação Dinâmica (DP), cujo custo de memória $O(W \times V)$ torna-se proibitivo para instâncias de larga escala. Foram avaliadas variantes da heurística gulosa (baseadas em eficiência linear e pseudo-utilidade, seguidas pelo refinamento via Busca Local (Hill Climbing) e pela metaheurística GRASP (Greedy Randomized Adaptive Search Procedure). Os experimentos conduzidos por oito instâncias de teste (até 1000 itens) revelaram que a Busca Local é altamente dependente da qualidade da solução inicial, atingindo apenas 47,75% do ótimo em inícios aleatórios na instância de maior escala. Em contrapartida, o GRASP consolidou-se como a estratégia mais robusta, mantendo lucros consistentes acima de 99% do ótimo absoluto, com um speedup massivo de até 1,1 milhão de vezes em relação a DP. Conclui-se que a hibridização do método guloso com*

o GRASP representa a solução ideal para sistemas de decisão em tempo real, equilibrando lucratividade máxima e rapidez operacional.

1. Introdução

A tomada de decisões sob limitações de recursos é um desafio constante tanto na vida cotidiana quanto em problemas complexos de engenharia. Imagine a necessidade de carregar uma mochila para uma viagem: o objetivo é escolher os itens mais valiosos ou úteis, mas existe um limite físico de quanto peso se pode carregar. Na ciência da computação, essa situação é modelada pelo Problema da Mochila (Knapsack Problem - KP), descrito por Martello e Toth (1990) como um dos desafios mais emblemáticos da otimização combinatória.

Embora o modelo clássico seja amplamente estudado, os cenários reais de logística e manufatura frequentemente exigem a observância de múltiplas limitações físicas simultâneas. Como explicam Guerrouma e Aïder (2025) e Soufyane, Karim e Chakir (2024), surge a necessidade de modelos mais robustos como o Problema da Mochila Bidimensional (2D-0/1 KP), no qual a seleção de itens deve respeitar não apenas um limite de peso, mas também uma restrição de volume ou espaço físico disponível.

A grande dificuldade em se trabalhar com o 2DKP reside na sua classificação como um problema NP-Difícil (GAREY; JOHNSON, 1979; FERNANDO; BALLERA, 2024). Conforme discutido por Fernando e Ballera (2024), isso significa que encontrar a solução ótima absoluta torna-se computacionalmente exaustivo a medida que a escala da instância cresce, exigindo tempo e memória que superam a capacidade de computadores convencionais. Métodos exatos, como a Programação Dinâmica, tornam-se proibitivos em instâncias de grande porte devido ao custo exponencial de processamento. Para superar essa barreira, pesquisadores como Le (2026) defendem o uso de heurísticas, estratégias de busca que priorizam a viabilidade prática em sistemas de tempo real ao entregar soluções de alta qualidade em frações de segundo.

Entre essas estratégias, a heurística guloso (Greedy Heuristic) destaca-se por sua eficiência e simplicidade de implementação em hardware com recursos limitados. A lógica fundamental de um algoritmo guloso eficiente, segundo Rezoug, Bader-El-Den e Boughaci (2019), repousa sobre o Princípio de Otimalidade Aproximada, que sugere que os melhores itens tendem a possuir alta eficiência entre lucro e custo. Complementando essa ideia Le (2026) e Kamal et al. (2025) propõe a ordenação dos itens por uma pseudo-utilidade, garantindo que o algoritmo priorize recursos que ofereçam o maior ganho marginal. No entanto, os métodos gulosos são inerentemente “míopes” e podem estagnar em ótimos locais pobres. Para mitigar essa limitação, este trabalho incorpora o refinamento via Busca Local (*Hill Climbing*) e a metaheurística GRASP (do Inglês, *Greedy Randomized Adaptative Search Procedure*), que utiliza a aleatoriedade controlada para explorar o espaço de busca de forma mais robusta (ARINA; RABADI, 2016; GUERROUMA; AÏDER, 2025).

Este artigo apresenta a implementação e análise comparativa de uma heurística gulosa desenvolvida em linguagem C, integrada a algoritmos de refinamento e busca estocástica para resolver o 2D-01 KP, com instâncias geradas de forma pseudo-aleatória. O estudo avalia desde a sensibilidade das funções de pontuação (*scores*) até o impacto das soluções iniciais no desempenho da Busca Local e do GRASP, visando o equilíbrio entre

lucratividade e rapidez operacional exigido pela engenharia contemporânea (LE, 2026; FERNANDO; BALLERA, 2024).

2. Revisão Bibliográfica

O presente capítulo estabelece a fundamentação teórica necessária para a compreensão do Problema da Mochila e de suas derivações multidimensionais. Segundo Glover e Kochenberger (2003), a evolução das técnicas de otimização permitiu que problemas clássicos fossem revisitados sob a ótica de metaheurísticas robustas, capazes de lidar com espaço de busca massivos. Serão explorados os aspectos de taxonomia e complexidade computacional que justificam a migração dos métodos exatos para abordagens heurísticas e estocásticas, bem como os princípios de vizinhança e aleatoriedade controlada que sustentam a implementação da Busca Local e do GRASP neste estudo.

2.1. Taxonomia e Complexidade

O Problema da Mochila (Knapsack Problem - KP) é amplamente reconhecido na literatura como um dos modelos mais emblemáticos da otimização combinatória (MARTELLO; TOTH, 1990). Sua essência reside na tomada de decisão binária para maximizar lucros sob restrições de capacidade, servindo como base para desafios que variam desde o planejamento orçamentário até a logística avançada (MARTELLO; TOTH, 1990; GUERROUMA; AÏDER, 2025). Contudo, a engenharia contemporânea exige modelos mais robustos, como o Problema da Mochila Multidimensional (MdKP).

O 2D-0/1 KP, foco deste estudo, situa-se na família dos MdKP de vetores, onde cada item é caracterizado por duas funções de peso independentes (tipicamente peso e volume) e uma restrição de capacidade imposta individualmente a cada função (CACCHIANI et al., 2022; SOUFYANE; KARIM; CHAKIR, 2024). Diferente do Problema da Mochila Geométrico, que lida com o empilhamento físico de caixas sem sobreposição, o 2DKP vetorial foca no consumo de recursos escassos simultâneos (CACCHIANI et al., 2022).

A relevância acadêmica do 2DKP reside na sua classificação como um problema NP-difícil no sentido forte (GAREY; JOHNSON, 1979; MARTELLO; TOTH, 1990). Conforme discutido por Fernando e Ballera (2024), essa classificação implica que o espaço de busca cresce de forma exponencial em relação ao número de itens N , tornando a busca pela solução ótima global um desafio computacional exaustivo. Em instâncias de grande porte, o uso de Programação Dinâmica torna-se inviável devido ao custo de memória $O(W \times V)$, onde W e V representam as capacidades máximas de cada dimensão (MARTELLO; TOTH, 1990; FERNANDO; BALLERA, 2024).

Além da barreira matemática, a estrutura do 2D-0/1 KP reflete limitações físicas reais de armazenamento. Le (2026) destaca que, em sistemas de engenharia e redes de sensores, raramente um item é limitado apenas por sua massa; o espaço físico ocupado (volume) é frequentemente o gargalo primário. Assim, a formalização deste problema busca espelhar a complexidade de ambientes onde o desperdício de qualquer uma das dimensões de recurso resulta em ineficiência críticas para o sistema (SOUFYANE; KARIM; CHAKIR, 2024; KAMAL et al., 2025).

2.2. O Conflito entre Métodos Exatos e Heurísticos

Historicamente, a busca pela solução do Problema da Mochila fundamentou-se em métodos exatos como a Programação Dinâmica (do Inglês, Dynamic Programming - DP) e o Branch and Bound (B&B) (MARTELLO; TOTH, 1990; FERNANDO; BALLERA, 2024). Embora esses algoritmos garantam a otimalidade absoluta através de uma exploração sistemática de todas as combinações de subproblemas, a classificação do 2D-0/1 KP como um problema NP-difícil impõe uma barreira computacional intransponível para instâncias de larga escala (CACCHIANI et al., 2022; GAREY; JOHNSON, 1979). Como observado por Fernando e Ballera (2024), a medida que o número de itens N cresce, o tempo de processamento para métodos exaustivos torna-se proibitivo, falhando em fornecer respostas em tempos úteis para a indústria.

No caso específico da DP aplicada ao problema bidimensional, o desafio não é apenas o tempo, mas também o consumo de memória, que se eleva para uma ordem de complexidade de $O(W \times V)$ (LE, 2026). Para sistemas reais com restrições de peso (W) e volume (V) elevadas, o armazenamento da matriz de estados pode exceder a capacidade de memória RAM disponível, consolidando o fenômeno conhecido como “explosão combinatória” (FERNANDO; BALLERA, 2024; LE, 2026). Essa limitação técnica justifica a migração para métodos aproximados que priorizam a viabilidade prática.

Nesse cenário, pesquisadores como Le (2026) e Soufyane, Karim e Chakir (2024) defendem que a rapidez na tomada de decisão é, muitas vezes, mais valiosa do que a garantia do lucro máximo teórico, especialmente em dispositivos de borda com recursos de hardware limitados. A necessidade de balancear o rigor matemático com a agilidade operacional é um tema recorrente na Engenharia de Computação moderna. Conforme enfatizado por Fernando e Ballera (2024), *frameworks* que utilizam heurísticas leves conseguem manter um desempenho próximo ao ótimo (frequentemente com qualidade superior a 95%), permitindo que o sistema responda dinamicamente a variações de carga sem sobrecarregar o processador central.

Complementando essa visão, Arina e Rabadi (2016) discute que até mesmo metaheurísticas complexas podem sofrer com altos custos computacionais devido às suas extensas fases de refinamento. A tendência atual inclina-se para o desenvolvimento de “algoritmos enxutos”, que substituem processamentos pesados por lógicas de busca direcionadas e eficientes (ARINA; RABADI, 2016). Este trabalho adota essa premissa ao hibridizar o Método Guloso com a Busca Local e o GRASP, buscando transpor a barreira da complexidade NP-difícil através de um equilíbrio robusto entre lucratividade e velocidade de execução (LE, 2026; GUERROUMA; AÏDER, 2025).

2.3. Fundamentos da Heurística Gulosa e Critérios de Ordenação

A heurística gulosa fundamenta-se na premissa da “ganância local”, na qual o algoritmo seleciona o melhor item disponível em cada etapa de construção, sem reavaliar decisões passadas ou prever consequências globais de longo prazo (SOUFYANE; KARIM; CHAKIR, 2024). A simplicidade dessa lógica permite uma implementação de baixo custo operacional, sendo ideal para sistemas que exigem execução imediata em *hardware* limitado (LE, 2026). Entretanto, a qualidade da solução final está intrinsecamente ligada ao critério de ordenação escolhido para priorização dos itens (MARTELLO; TOTH, 1990; SOUFYANE; KARIM; CHAKIR, 2024).

Segundo Rezoug, Bader-El-Den e Boughaci (2019), a eficácia desses métodos repousa sobre o Princípio de Otimalidade Aproximada (POP). Este conceito sugere que, em problemas de mochila, as soluções de alta performance tendem a ser compostas por itens que apresentam uma alta eficiência relativa entre lucro e consumo de recursos. Dessa forma, a ordenação dos itens por uma métrica de “eficiência” busca colocar no topo da lista os elementos que têm maior probabilidade de pertencer ao ótimo global (REZOU; BADER-EL-DEN; BOUGHACI, 2019).

Para superar a “miopia” inerente ao método, especialmente no contexto bidimensional onde peso e volume competem pelo espaço, aplicam-se critérios de Densidade de utilidade ou Pseudo-utilidade (LE, 2026; KAMAL et al., 2025). Como discutido por Le (2026), em problemas multidimensionais, o *score* de um item não deve ser apenas o lucro absoluto, mas uma métrica composta que penalize o consumo de qualquer recurso escasso. *Frameworks* modernos sugerem a normalização das restrições para evitar que uma dimensão (ex: peso) domine a outra (ex: volume) injustamente no processo de seleção (LE, 2026; SOUFYANE; KARIM; CHAKIR, 2024).

Neste trabalho, a função de *score* implementada segue essa filosofia de eficiência espacial. Ao transformar o consumo de peso e volume em frações da capacidade total da mochila, o algoritmo busca unificar a complexidade de restrições independentes em um único vetor de decisão linear (LE, 2026). Essa estratégia de ordenação por eficiência é tão robusta que serve de base não apenas para algoritmos isolados, mas também para a inicialização de populações em metaheurísticas mais complexas, como demonstrado nos estudos de Rezoug, Bader-El-Den e Boughaci (2019) e Arina e Rabadi (2016).

2.4. Aplicações Práticas e Hibridizações Metaheurísticas

A versatilidade da heurística gulosa permite que ela atue não apenas como um método isolado de busca, mas como um componente vital em sistemas híbridos de alta performance. De acordo com Rezoug, Bader-El-Den e Boughaci (2019), Guerrouma e Aïder (2025) e Qin et al. (2025), algoritmos gulosos são amplamente empregados para a inicialização de populações em metaheurísticas genéticas e meméticas. Ao fornecer uma base de soluções iniciais já qualificadas, a heurística reduz significativamente o tempo necessário para a convergência evolutiva, impedindo que o algoritmo desperdice recursos explorando áreas indesejadas do espaço de soluções.

No campo da engenharia, as aplicações práticas desses modelos são diversas e destacam a necessidade de algoritmos de resposta rápida. Por exemplo, Prommakhot, Arimura e Thoumeun (2025) utiliza heurísticas gulosas como parte de um decodificador para otimizar a distribuição de carregamento elétrico, enquanto Mazumdar e Deka (2025) aplica lógicas similares para o agendamento eficiente em redes IoT industriais. A capacidade de entregar resultados próximos ao ótimo em frações de segundo torna essas técnicas ideais para ambientes dinâmicos que não comportam o tempo de processamento de métodos exatos.

3. O Problema da Mochila

Neste capítulo, apresenta-se a formalização matemática do problema abordado, estabelecendo os limites operacionais e os critérios de viabilidade para a seleção dos itens. O problema em questão é classificado por Cacchiani et al. (2022) como uma variação do

MdKP, especialmente o 2D-0/1 KP vetorial. Segundo Cacchiani et al. (2022), esta variante caracteriza-se por possuir funções de peso independentes – neste caso, peso e volume – com restrições de capacidade impostas individualmente a cada dimensão, diferindo dos problemas de mochila geométricos que lidam com o empilhamento físico de objetos sem sobreposição.

3.1. Definição dos Parâmetros

De acordo com a implementação estruturada no código-fonte e as convenções adotadas em estudos contemporâneos de otimização (SOUFYANE; KARIM; CHAKIR, 2024; GUERROUMA; AÏDER, 2025), definem os seguinte componentes matemáticos:

- Seja N o número total de itens disponíveis para seleção (referenciadas como `n` no código);
- Seja i o índice que representa um item específico de 1 a N ;
- Seja p_i o lucro (ou valor) associado ao item i (definido como `lucro` no código);
- Seja w_i o peso do item i (definido como `peso` no código);
- Seja v_i o volume do item i (definido como `volume` no código);
- Sejam W e V as capacidades máximas de peso e volume da mochila, respectivamente (referenciadas como `maxPeso` e `maxVolume` no código).

3.2. Variáveis de Decisão e Função Objetivo

A natureza “0-1” do problema impede o fracionamento dos itens, exigindo uma decisão binária de inclusão ou exclusão (ARINA; RABADI, 2016; SOUFYANE; KARIM; CHAKIR, 2024). Para modelar esse comportamento, utiliza-se a variável de decisão x_i definida como:

$$x_i = \begin{cases} 1, & \text{se o item } i \text{ for incluído na mochila} \\ 0, & \text{caso contrário} \end{cases} \quad (1)$$

O objetivo primordial do algoritmo consiste em encontrar um subconjunto de itens que maximize o lucro total acumulado (Z), conforme expresso pela função de objetivo abaixo:

$$\text{Maximizar } Z = \sum_{i=1}^N p_i x_i \quad (2)$$

3.3. Restrições de Viabilidade

Para que uma solução seja considerada válida, o somatório do consumo de recursos não pode exceder os limites físicos estabelecidos. Assim, o modelo deve respeitar simultaneamente as seguintes condições de contorno (LE, 2026; CACCHIANI et al., 2022):

- **Restrição de Peso:** O somatório dos pesos dos itens escolhidos não pode ultrapassar a capacidade W .

$$\sum_{i=1}^N w_i x_i \leq W \quad (3)$$

- **Restrição de Volume:** O somatório dos volumes dos itens escolhidos não pode ultrapassar a capacidade V .

$$\sum_{i=1}^N v_i x_i \leq V \quad (4)$$

- **Domínio da Variável:** Garante a integridade da decisão binária (Mochila 0-1).

$$x_i \in \{0, 1\}, \quad \forall i \in \{1, \dots, N\} \quad (5)$$

4. Heurística Construtiva: O Método Guloso

A inviabilidade de métodos exatos para instâncias de larga escala do 2D-0/1 KP, conforme discutida no Capítulo 2, exige a adoção de estratégias que priorizem a eficiência computacional. O paradigma da heurística gulosa surge como a abordagem mais direta para este desafio. Conforme descrito por Soufyane, Karim e Chakir (2024), esse paradigma opera sob o princípio de tomar decisões locais ótimas em cada etapa da construção da solução, sem reavaliar escolhas passadas ou prever consequências globais de longo prazo.

Apesar da natureza “míope”, algoritmos gulosos são componentes fundamentais na otimização contemporânea. Segundo Martello e Toth (1990), eles oferecem uma base sólida para construção de soluções iniciais que podem ser posteriormente refinadas por metaheurísticas mais robustas. Neste trabalho, o método guloso é implementado como o primeiro estágio de busca, visando preencher a capacidade da mochila de forma rápida e rentável através de critérios de eficiência espacial.

4.1. Métrica de Avaliação: Score Inicial

O núcleo operacional de um algoritmo guloso reside na sua função de priorização, que deve ser capaz de identificar quais itens oferecem o melhor “custo-benefício” antes de iniciar a fase de inserção (SOUFYANE; KARIM; CHAKIR, 2024). Enquanto no problema da mochila clássico a ordenação baseia-se em uma razão direta entre lucro e peso, o 2D-0/1 KP exige a síntese de duas restrições físicas independentes em um único escalar de decisão (CACCHIANI et al., 2022).

A primeira estratégia adotada neste estudo, implementada na função `calcularScoreSimples`, utiliza uma abordagem de eficiência linear. Esse critério assume que a atratividade de um item é inversamente proporcional ao consumo total bruto de seus recursos. Matematicamente o score inicial do item i é definido pela Equação 6:

$$Score_i = \frac{p_i}{w_i + v_i} \quad (6)$$

Onde p_i representa o lucro, w_i o peso e v_i o volume do item. Conforme discutido por Rezoug, Bader-El-Den e Boughaci (2019), essa métrica fundamenta-se no POP, operando sob a premissa de que itens com alta densidade de lucro por unidade de recurso tem uma probabilidade estatística superior de compor o subconjunto ótimo global.

Apesar de sua simplicidade e baixo custo computacional ($O(1)$ por item), esta métrica linear carrega uma limitação teórica importante: ela negligencia a escassez relativa das capacidades da mochila (SOUFYANE; KARIM; CHAKIR, 2024). Ao somar peso e volume sem qualquer ponderação, o algoritmo pressupõe que uma unidade de

massa equivale exatamente a uma unidade de espaço. Como alertado por Le (2026) e observado em seus experimentos práticos, essa equivalência falha em instâncias assimétricas ($W \neq V$), onde a restrição mais “apertada” (o gargalo do sistema) deveria exercer uma pressão maior no cálculo do *score* para evitar a subutilização de recursos críticos. Essa insuficiência justifica a necessidade de uma métrica normalizada, detalhada na seção seguinte.

4.2. Métrica Refinada: Pseudo-utilidade e Normalização

Para mitigar as distorções causadas pela soma linear de grandezas heterogêneas, este trabalho propõe uma segunda métrica de ordenação fundamentada no conceito de Pseudo-utilidade ou Densidade de Utilidade, conforme discutido por Le (2026) e Kamal et al. (2025). A premissa central desta abordagem é que, em problemas de mochila multidimensional, a atratividade de um item deve ser avaliada em relação a escassez relativa de cada recurso na mochila, evitando que a dimensão com maior capacidade nominal domine indevidamente o critério de seleção (LE, 2026; SOUFYANE; KARIM; CHAKIR, 2024).

A métrica refinada, implementada na função `calcularScore` do algoritmo desenvolvido introduz os parâmetros de normalização α (alfa) e β (beta) para ponderar o consumo de peso (w_i) e volume (v_i). O *score* do item i é redefinido pela Equação 7:

$$Score_i = \frac{p_i}{\alpha w_i + \beta v_i} \quad (7)$$

Nesta formulação, os coeficientes de ponderação são definidos como o inverso das capacidades totais sendo $\alpha = 1/W$ e $\beta = 1/V$ (LE, 2026). Ao aplicar esses fatores, o algoritmo passa a processar o consumo de recursos como frações da capacidade total disponível. Como destacam Soufyane, Karim e Chakir (2024) e Le (2026), essa transformação permite que o critério de ranqueamento identifique o “gargalo” do sistema: se a mochila possuir um volume muito limitado em relação ao peso, o coeficiente β exercerá uma penalidade maior sobre itens volumosos, preservando o espaço para elementos de maior eficiência espacial.

Esta estratégia de normalização busca conferir à heurística gulosa uma sensibilidade adaptativa à geometria de cada instância. Em vez de uma ganância linear cega, a pseudo-utilidade orienta a construção da solução para um equilíbrio que maximize o lucro marginal enquanto gerencia de forma inteligente a exaustão dos recursos mais críticos da mochila (SOUFYANE; KARIM; CHAKIR, 2024).

4.3. O Algoritmo Implementado

O processo de execução da função `mochilaGulosa`, estruturado em linguagem C, segue um passo a passo rigoroso para garantir a viabilização da solução final:

- **Cálculo de Score:** O algoritmo percorre a lista de N itens e calcula o score de cada um utilizando a métrica refinada de pseudo-utilidade;
- **Ordenação:** Os itens são ordenados de forma decrescente com base no score calculado. No código, isso é realizado através da função `qsort` e da função auxiliar `comparar`, garantindo que os itens de maior “custo-benefício” fiquem no topo da lista.

- **Seleção e Inserção:** O algoritmo percorre a lista ordenada e, para cada item, verifica se a sua inclusão não viola as restrições físicas de peso e volume definidas no modelo matemático.
- **Atualização de Estados:** Se o item couber, ele é adicionado a mochila, e as capacidades são atualizadas subtraindo-se w_i e v_i . O processo se repete até que a lista termine ou a mochila esteja saturada.

Essa estratégia reflete a aplicação prática da "ganância" técnica: o algoritmo olha para os itens, calcula o que mais vale a pena de acordo com o espaço relativo que ocupa e insere os elementos até que os limites físicos impeçam novas adições.

5. Heurística de Refinamento: Busca Local

Embora os algoritmos construtivos baseados em estratégias gulosas apresentem alta eficiência computacional, a tomada de decisão localmente ótima e "miope" frequentemente conduz o processo de busca à estagnação em ótimos locais subestimados em relação ao potencial da instância (ARINA; RABADI, 2016). Como destacado por Arina e Rabadi (2016), a maioria das heurísticas eficazes para problemas de otimização combinatória discreta é estruturada em duas fases consecutivas: uma fase de construção, responsável por gerar a viabilidade inicial, seguida por uma fase de melhoria ou refinamento via Busca Local.

No contexto do 2D-0/1 KP, o refinamento atua rearranjando os elementos selecionados para explorar espaços residuais de capacidade e elevar a lucratividade total (GUERROUMA; AÏDER, 2025). Segundo a metáfora do *Hill Climbing* (do Português, Subida de Encosta) discutida por Glover e Kochenberger (2003), o algoritmo navega pelo "cenário de *fitness*" realizando transições sucessivas para soluções vizinhas de melhor qualidade até atingir o topo de um vale de convergência, onde não existem mais melhorias imediatas possíveis dentro da vizinhança definida.

5.1. Formulação Matemática do Espaço de Vizinhança

Para formalizar a mecânica de refinamento implementada na função `aplicarBuscaLocal`, define-se uma solução corrente do 2D-0/1 KP por um vetor de decisão $z = (x_1, x_2, \dots, x_n)$, onde $x_i = 1$ se o item i está inserido na mochila e $x_i = 0$ caso contrário (GLOVER; KOCHENBERGER, 2003). Seja S o conjunto de índices dos itens atualmente selecionados, o peso total $W(S)$ e o volume total $V(S)$ consumidos são dados pelo somatório de seus respectivos recursos w_i e v_i (ARINA; RABADI, 2016).

A estrutura de vizinhança adotada neste estudo baseia-se em movimentos de troca simples, que buscam substituir um recurso interno por outro externo mais vantajoso. O espaço de vizinhança $N(x)$ de uma solução corrente x é composto por todas as soluções viáveis x' que podem ser alcançadas ao remover exatamente um item $i \in S$ e inserir exatamente um item $j \notin S$ (ARINA; RABADI, 2016; GUERROUMA; AÏDER, 2025). Conforme estabelecido no algoritmo desenvolvido, uma solução vizinha x' é considerada válida e candidata à transição se satisfizer simultaneamente os critérios fundamentais:

- **Viabilidade de Peso:** $(W(s) - w_i + w_j) \leq W;$
- **Viabilidade de Volume:** $(V(S) - v_i + v_j) \leq V;$

- **Critério de Melhoria Estrita:** $p_j > p_i$.

Esta restrição de que o lucro do item entrante (p_j) deve ser estritamente superior ao do item removido (p_i) garante que cada movimento de vizinhança resulte em um incremento real na função objetivo Z (ARINA; RABADI, 2016). Essa abordagem transforma a busca em uma exploração direcionada do espaço de soluções, filtrando vizinhos que degradariam o lucro acumulado, focando o esforço computacional apenas em trajetórias de ascensão de valor.

5.2. Estratégia de Exploração e Condição de Aceitação

A exploração do espaço de vizinhança $N(x)$ foi estruturada sob o paradigma da Primeira Melhoria (do Inglês, *First-Improvement*), um critério de aceitação de movimentos que prioriza a agilidade na transição de estados (GLOVER; KOCHENBERGER, 2003). Diferente da estratégia de Melhor Melhoria (do Inglês, *Best-Improvement*), que exige a varredura completa de toda a vizinhança para selecionar o movimento ótimo, a Primeira Melhoria consolida a troca de itens assim que o primeiro vizinho viável que satisfaça os critérios de lucro e capacidade é identificado.

Conforme implementado no algoritmo desenvolvido, o processo opera por meio de dois laços aninhados que percorrem a lista de itens N . O laço externo identifica elementos já selecionados, enquanto o laço interno busca candidatos externos que ofereçam lucro superior ($p_j > p_i$). No momento em que uma troca válida é detectada, o algoritmo interrompe a varredura atual e sinaliza a ocorrência de progresso por meio de uma *flag*.

Essa interrupção imediata seguida pelo reinício da busca a partir do primeiro item da lista é fundamental para a dinâmica do refinamento. Essa abordagem permite que o algoritmo aproveite rapidamente o espaço residual liberado pela troca, reavaliando todos os itens sob nova configuração de capacidade da mochila.

A execução encerra-se apenas quando uma varredura completa é realizada sobre todos os pares (i, j) sem que nenhuma troca válida seja identificada. neste estágio, a *flag* permanece em nível lógico zero, indicando que o algoritmo atingiu um Ótimo Local estável em relação a vizinhança de troca simples, esgotando as possibilidades de incremento direto na função objetivo Z dentro do escopo definido.

6. Metaheurística GRASP

Como observado nas seções anteriores, os algoritmos construtivos gulosos são rápidos, mas limitados a uma única trajetória determinística, enquanto a Busca Local é fortemente dependente da qualidade do ponto de partida (ARINA; RABADI, 2016). Para superar essas limitações e permitir uma exploração mais abrangente do espaço de busca do Problema da Mochila Bidimensional 0-1, implementou-se a metaheurística GRASP (*Greedy Randomizes Adaptive Search Procedure*). Conforme proposto originalmente na literatura e discutido por Arina e Rabadi (2016), o GRASP é um processo iterativo de dois estágios que combina de forma sistemática uma fase de construção gulosa randomizada e uma fase de intensificação via busca local.

6.1. Fase de Construção Randomizada e Lista de Candidatos Restrita (LCR)

Diferente de um algoritmo construtivo puramente guloso, que seleciona sempre o item com maior *score* de eficiência, a fase de construção do GRASP introduz variabilidade

no processo de escolha através da criação de uma Lista de Candidatos Restrita (LCR). A cada etapa da construção, avaliam-se todos os itens que ainda não foram selecionados e que respeitam as restrições de peso e volume residual da mochila.

Seja s_{max} o maior *score* de eficiência encontrado entre os itens candidatos viáveis e s_{min} o menor *score* observado nessa mesma etapa, a inclusão de um item i na LCR é controlada por um parâmetro de calibração alfa (α), cujo intervalo está contido entre 0 e 1 ($0 \leq \alpha \leq 1$). Um item i é considerado um candidato elegível para compor a LCR se o seu *score* individual s_i atender ao seguinte critério matemático:

$$s_i \geq s_{max} - \alpha(s_{max} - s_{min}) \quad (8)$$

O parâmetro alfa atua como o regulador do balanço entre ganância e aleatoriedade no algoritmo. Quando alfa é igual a 0, o critério torna-se estritamente guloso, pois apenas os itens com o *score* máximo são admitidos na lista. Por outro lado, se alfa é igual a 1, a restrição é completamente relaxada, permitindo que todos os itens viáveis (até mesmo aqueles com s_{min}) entrem na LCR, transformando a construção em uma amostragem puramente aleatória.

Uma vez estruturada a LCR com todos os elementos que satisfazem o limite de corte, o algoritmo realiza um sorteio aleatório uniforme para determinar qual item será efetivamente inserido na mochila. As capacidades de peso e volume consumidas são atualizadas, os *scores* dos itens restantes são recalculados adaptativamente e o processo se repete até que a LCR fique vazia, indicando que nenhum item adicional consegue preencher os espaços residuais sem violar os limites físicos da mochila.

6.2. Fase de Refinamento (Busca Local)

A solução inicial gerada pela fase de construção gulosa randomizada, embora viável e diversificada, raramente coincide com um ótimo local de alta performance estatística (ARINA; RABADI, 2016; GUERROUMA; AÏDER, 2025). Por essa razão, a metaheurística acopla imediatamente o procedimento de Busca Local detalhado no Capítulo 5 como um estágio de intensificação obrigatório (ARINA; RABADI, 2016).

O diferencial crítico desta integração reside na natureza probabilística do ponto de partida: como a solução inicial possui uma componente estocástica definida pelo parâmetro α , cada iteração do GRASP fornece à Busca Local uma “coordenada” distinta no espaço de soluções (GUERROUMA; AÏDER, 2025).

Essa amostragem múltipla permite que o refinamento escape de ótimos locais pobres e explore diferente vales de convergência dentro do cenário de *fitness* da instância. Enquanto um algoritmo guloso determinístico ficaria preso a uma única trajetória míope, a estrutura iterativa do GRASP possibilita que o sistema neutralize a rigidez do problema, garantindo que o refinamento atue sobre diversas regiões promissoras do espaço de busca (ARINA; RABADI, 2016; GUERROUMA; AÏDER, 2025). Conforme defendido por Arina e Rabadi (2016), essa simbiose entre construção diversificada e intensificação direcionada é o que confere ao GRASP a robustez necessária para atingir resultados próximos à otimalidade absoluta em problemas multidimensionais complexos.

6.3. Estrutura Multi-start e Critério de Parada

Por se tratar de um algoritmo probabilístico, o GRASP opera sob uma arquitetura de múltiplos inícios (*multi-start*). O ciclo completo – composto por construção gulosa randomizada seguida por refinamento – é encapsulado dentro de um laço de repetição delimitado por um número pré-definido de iterações máximas.

Ao término de cada iteração, o lucro total da solução refinada atual é comparado com o lucro da melhor solução armazenada até aquele momento pelo sistema. Se a nova solução apresenta um valor científico-financeiro estritamente superior, o algoritmo atualiza o registro da melhor solução global, salvando a configuração dos itens selecionados, o peso final e o volume final consumidos. O critério de parada adotado nesta implementação é determinístico, baseado no esgotamento do número de iterações estipulado pelo usuário. Ao atingir o limite, a metaheurística encerra sua execução e retorna a melhor solução viável encontrada ao longo de toda a sua trajetória de amostragem.

7. Experimentos Computacionais

Essa seção descreve a metodologia aplicada para avaliar o desempenho dos algoritmos desenvolvidos, detalhando o processo de criação das instâncias de teste e a obtenção dos resultados de referência para comparação.

Para validar a eficácia das estratégias propostas, foi desenvolvida uma bateria de testes automatizada, estruturada para simular diferentes cenários de carga e restrições. O ambiente experimental utiliza funções de alta precisão para medições de tempo de execução, garantindo a confiabilidade dos dados coletados em milissegundos.

7.1. Geração de Instâncias Pseudo-aleatórias

A construção dos dados de teste foi realizada por gerador customizado que busca imitar a complexidade de problemas reais. Foram criados oito casos de teste, divididos entre capacidade simétricas ($W = V$) e assimétricas ($W \neq V$), variando de 50 a 1000 itens.

A lógica de geração de itens no código `gerarItensAleatorios` utiliza duas categorias distintas:

- **Itens de Ouro:** Representam aproximadamente 5% do total e possuem peso e volume reduzido (entre 1% e 2% da capacidade máxima), mas com lucros desproporcionalmente elevados. Esses itens testam a capacidade da heurística em priorizar elementos de alta eficiência.
- **Itens Regulares:** Possuem lucro baseado no consumo total de recursos (soma de peso e volume), acrescido de um "ruído" aleatório que varia de -10% a 20% do valor base ($tamanho_{base} = itens[i].peso + itens[i].volume$). Essa correlação entre lucro e recurso aumenta a dificuldade do problema aproximando-se de cenários de *strong correlation* discutidos na literatura (KAMAL et al., 2025).

7.2. Resolução por Programação Dinâmica (Ótimo)

Para estabelecer um limite superior de qualidade, as instâncias são resolvidas através de um algoritmo de Programação Dinâmica (DP) implementado na função `resolverDP`. Conforme sugerido por (FERNANDO; BALLERA, 2024), a DP garante a obtenção da

solução ótima global ao explorar sistematicamente todas as combinações possíveis de subproblemas.

Matematicamente, o algoritmo constrói uma matriz tridimensional implícita (embora otimizada para duas dimensões de recurso no código) onde cada estado $dp[w][v]$ armazena o lucro máximo possível para capacidades parciais w e v . Apesar de exigir um custo de memória elevado ($W \times V$), essa abordagem é indispensável para calcular o *gap* de otimalidade das heurísticas propostas.

7.3. Aplicação dos Métodos Gulosos

Com base de comparação estabelecida pela DP, o experimento aplica duas variantes da heurística gulosa para medir o equilíbrio entre tempo e qualidade:

- **Guloso Simples:** Utilizando métricas de score baseada na soma linear dos recursos:

$$score = \frac{p_i}{(w_i + v_i)} \quad (9)$$

Esta abordagem é extremamente rápida, mas pode falhar em instâncias onde as capacidades de peso e volume são muito distintas, pois não normaliza as grandezas.

- **Guloso Otimizado (Pseudo-utilidade):** Aplica a métrica refinada com pesos α e β baseado nas capacidades totais da mochila. Ao transformar o consumo de cada item em uma porcentagem da capacidade disponível, o algoritmo torna-se mais "sensível" a restrição mais escassa, permitindo uma seleção de itens mais estratégica em cenários de assimetria.

Contrariando a expectativa teórica inicial de que a normalização sempre traria benefícios, os resultados experimentais revelam um comportamento distinto conforme a natureza da instância. Nas instâncias simétricas de grande escala (Instâncias 3 e 4), o Guloso Otimizado mostrou-se superior, atingindo qualidades de até 99,63%. Entretanto, em quase todos os casos de instância assimétricas (Instâncias 5 a 8), o Guloso Otimizado apresentou um desempenho inferior ao Guloso Simples.

Esse fenômeno sugere que, ao tentar equilibrar as dimensões de peso e volume através da pseudo-utilidade em cenários assimétricos, a heurística pode estar superestimando o custo de itens que seriam vantajosos sob uma análise linear simples, levando a uma subutilização de um dos recursos e, consequentemente, a um lucro total menor em relação ao método linear.

7.4. Análise do Impacto das Soluções Iniciais no Refinamento (Busca Local)

A eficácia de um algoritmo de Busca Local é intrinsecamente dependente da qualidade da solução inicial fornecida ao sistema. Como discutido na literatura de metaheurísticas, o ponto de partida define em qual "vale" de convergência o algoritmo irá operar, podendo facilitar ou dificultar o alcance de regiões de alta lucratividade no espaço de busca. Nesta subseção, analisam-se os resultados obtidos pela estratégia de *Hill Climbing* quando inicializada por duas fontes distintas: uma construção puramente aleatória (Aleatório + BL) e a base qualificada da heurística gulosa (Guloso + BL).

7.4.1. Qualidade da Solução e Escalabilidade:

Os dados da Tabela 1 revelam um comportamento distinto entre as duas abordagens de refinamento conforme a escala do problema aumenta. Em instâncias de pequeno porte (T1 e T5), a estratégia Aleatória + BL apresentou resultados competitivos, chegando a superar o refinamento guloso na instância T5, onde atingiu 98,85% de qualidade frente aos 95,43% do método híbrido. Esse fenômeno sugere que, em espaços de busca reduzidos, a aleatoriedade pode fornecer pontos de partida que, embora desestruturados, permitem a busca local explorar caminhos que a “ganância” linear ignora.

Contudo, ao avançar para instâncias de larga escala (T4 e T8), observa-se o colapso da eficácia baseada em iícios aleatórios. Na instância máxima (T8), o método Aleatório + BL obteve apenas 47,75% do ótimo, enquanto o Guloso + BL manteve uma performance robusta de 98,25%. A medida que o número de itens cresce, a probabilidade de um início aleatório estar em uma região “fértil” do espaço de soluções diminui drasticamente, tornando o refinamento insuficiente para corrigir a desordem inicial. Em contrapartida, o uso de uma solução inicial gulosa coloca o algoritmo de busca local em uma vizinhança já próxima do topo, permitindo que o refinamento atue apenas em ajuste finos e trocas estratégicas para elevar a qualidade para patamares superiores a 94% em todos os casos testados.

7.4.2. Eficiência Computacional de Convergência

No que tange ao tempo de execução, a Tabela 2 demonstra que o refinamento a partir de soluções gulosas é, em média, mais rápido ou equivalente em termos de esforço computacional em instâncias maiores (como T3), validando a premissa de que boas soluções iniciais exigem menos passos de melhoria para atingir o ótimo local.

Entretanto, em instâncias como a T4, o tempo do Guloso + BL (3,776 ms) foi maior ao do Aleatório + BL (1,139 ms). Esse aumento no tempo de processamento não indica ineficiência, mas sim a profundidade da exploração: o início guloso situa o algoritmo em uma região densa de soluções de alta qualidade, onde pares de trocas viáveis ($p_j > p_i$) são mais numerosos e complexos de processar antes da convergência final. Já a rapidez do início aleatório em casos de grande escala deve-se, muitas vezes, à estagnação precoce em ótimos locais pobres, onde o algoritmo rapidamente esgota as possibilidades de troca devido a violação severa das restrições de capacidade logo nas primeiras iterações.

Em suma, para o 2DKP, a hibridização do Método Guloso com a Busca Local prova ser a estratégia mais equilibrada para aplicações de engenharia. Ela garante que, independentemente da assimetria ou escala dos dados, o sistema entregue uma solução de alta confiabilidade em tempo quase instantâneo, superando a imprevisibilidade de métodos puramente estocásticos.

7.5. Desempenho e Calibração da Metaheurística GRASP

Com o objetivo de superar a rigidez das trajetórias determinísticas dos métodos gulosos e a sensibilidade da Busca Local aos pontos de partida, a metaheurística GRASP foi submetida a uma avaliação rigorosa. Conforme discutido na literatura, o diferencial do

GRASP reside na sua estrutura iterativa de dois estágios, que combina sistematicamente uma fase de construção gulosa randomizada com uma fase de intensificação via busca local.

7.5.1. Calibração do Parâmetro de Aleatoriedade (α)

A eficácia do GRASP no contexto do 2DKP é intrinsecamente ligada a calibração do parâmetro α (alfa), que regula a formação da LCR. Ao estabelecer que um item só é elegível se seu score estiver dentro de uma margem de qualidade em relação ao melhor candidato (Equação 8), o algoritmo evita o determinismo míope.

Durante os experimentos, observou-se que o GRASP atua como um mecanismo de amostragem estratégica do espaço de soluções: cada iteração fornece uma coordenada inicial distinta para a busca local, permitindo que o sistema escape de ótimos locais pobres e explore diferentes “vales” de convergência. Essa característica resolve o problema de estagnação precoce observado no método “Guloso + BL” em instâncias complexas.

7.5.2. Qualidade da Solução e Estabilidade Estatística

Os dados compilados na Tabela 1 confirmaram que o GRASP é a estratégia mais robusta entre todas as heurísticas testadas. Enquanto o desempenho do “Guloso + BL” oscilou significativamente em instâncias assimétricas (com na T7, onde atingiu 94,44%), o GRASP manteve uma qualidade superior a 99% em quase todos os cenários.

O destaque ocorre nas instâncias de máxima escala (T4 e T8). Na T8, onde o refinamento a partir de inícios aleatórios colapsou para apenas 47,75% do ótimo, o GRASP alcançou 99,23% de lucratividade. Esse resultado valida o Princípio de Otimalidade Aproximada (POP): ao combinar componentes de soluções de alta eficiência com uma busca estocástica, o GRASP consegue encontrar estruturas de solução muito próximas daquelas geradas pela Programação Dinâmica, mas com um custo computacional drasticamente menor.

7.5.3. Análise do Custo Computacional e Trade-off

Relacionado a eficiência, o GRASP apresentou, como esperado, os maiores tempo de execução entre as heurísticas devido a sua natureza *multi-start*. Entretanto, as medições da Tabela 2 reiteram sua viabilidade em sistemas de engenharia em tempo real. Mesmo na instância T4, o tempo de processamento de 34,094s representa um speedup de 7633x em relação à Programação Dinâmica (260,26s).

Essa performance, demonstra que é possível obter soluções com *gap* de otimalidade inferior a 1% de forma quase instantânea. Portanto o GRASP consolida-se neste trabalho como a abordagem de escolha para o 2DKP quanto a exigência por lucratividade máxima é tão crítica quando a necessidade de escalabilidade e rapidez operacional.

Tabela 1. Comparativo de Lucro Obtido e Qualidade da Solução em Relação ao Ótimo

Inst.	Dimensões ($m \times W \times V$)	DP (Ótimo)	Guloso Simples	Guloso Puro	Guloso + BL	Aleatório + BL	GRASP
T1	50 × 1000 × 1000	7542	6964 (92,34%)	6964 (92,34%)	7157 (94,90%)	7250 (96,13%)	7491 (99,32%)
T2	250 × 8000 × 8000	32175	28734 (89,31%)	28734 (89,31%)	30508 (94,82%)	25679 (79,81%)	31957 (99,32%)
T3	500 × 15000 × 15000	55914	55878 (99,94%)	55878 (99,94%)	55878 (99,94%)	48625 (86,96%)	55474 (99,21%)
T4	1000 × 23000 × 23000	89515	87778 (98,06%)	87778 (98,06%)	88601 (98,98%)	58920 (65,82%)	89087 (99,52%)
T5	50 × 800 × 1500	5157	5025 (97,44%)	4549 (88,21%)	4870 (94,43%)	5097 (98,84%)	5022 (97,38%)
T6	250 × 12000 × 4000	32627	29964 (91,84%)	29915 (91,69%)	31805 (97,48%)	25717 (78,82%)	32221 (98,76%)
T7	500 × 7000 × 18000	57003	56632 (99,35%)	52394 (91,91%)	53833 (94,44%)	41154 (72,20%)	56550 (99,21%)
T8	1000 × 23000 × 10000	83916	83829 (99,90%)	81056 (96,59%)	82450 (98,25%)	40067 (47,75%)	83270 (99,23%)

Tabela 2. Tempo de Execução e Ganho de Velocidade (*Speedup*) em Relação à DP

Inst.	Prog. Dinâmica (DP) (s)	Guloso Simples (ms)	Guloso Puro (ms)	Guloso + BL (ms)	Aleatório + BL (ms)	GRASP (ms)
T1	0,023387	0,010 (2338x)	0,004 (5846x)	0,018 (1271x)	0,026 (913x)	0,767 (30x)
T2	7,498684	0,046 (162k x)	0,030 (249k x)	0,288 (26k x)	0,154 (48k x)	7,711 (972x)
T3	55,06160	0,099 (555k x)	0,084 (653k x)	0,249 (221k x)	1,180 (46k x)	25,883 (2127x)
T4	260,2685	0,215 (1,2M x)	0,245 (1,0M x)	3,776 (68k x)	1,139 (228k x)	34,094 (7633x)
T5	0,029039	0,005 (5807x)	0,003 (10755x)	0,015 (1962x)	0,024 (1190x)	0,540 (53x)
T6	5,882228	0,024 (243k x)	0,019 (312k x)	0,334 (17k x)	0,076 (77k x)	4,255 (1382x)
T7	31,19702	0,050 (618k x)	0,042 (739k x)	1,290 (24k x)	0,452 (69k x)	16,706 (1867x)
T8	113,2942	0,103 (1,1M x)	0,099 (1,1M x)	0,874 (129k x)	0,587 (192k x)	32,142 (3524x)

Nota: Valores de *speedup* arredondados. Os sufixos **k** indicam milhares (ex: 162k x = 162.000 vezes mais rápido) e **M** indicam milhões de vezes mais rápido que a linha de base (DP).

8. Conclusão

Este estudo investigou estratégias para o Problema da Mochila Bidimensional 0-1, contrastando o rigor dos métodos exatos com a flexibilidade das abordagens heurísticas. A implementação em linguagem C permitiu validar que, embora a Programação Dinâmica forneça o teto de lucratividade, o seu custo exponencial de memória – da ordem de $O(W \times V)$ – atua como um limitador crítico para sistemas com recursos de hardware restritos ou instâncias de larga escala.

Uma contribuição central deste trabalho foi a análise da sensibilidade dos critérios de ordenação gulosos. Os experimentos revelam que a eficácia de uma métrica de *score* é geometricamente dependente: a pseudo-utilidade normalizada mostrou-se superior em cenários de alta escala simétrica, enquanto o cálculo linear simples exibiu maior resiliência em casos de forte assimetria entre peso e volume. Essa distinção sugere que, em aplicações industriais, o critério de priorização deve ser adaptado dinamicamente para identificar o gargalo específico da instância, evitando a subutilização de recursos valiosos.

No campo das heurísticas e metaheurísticas, observou-se que a qualidade do refinamento via Busca Local é intrinsecamente ligada a estrutura da solução inicial. O colapso estatístico observado em inícios puramente aleatórios em grandes instâncias reforça a necessidade de pontos de partida qualificados para evitar a estagnação precoce em ótimos locais pobres. Nesse cenário, o GRASP consolidou-se como a solução definitiva para o problema abordado. Ao equilibrar a ganância técnica com a exploração estocástica, a metaheurística neutralizou as variações de geometria das instâncias, garantindo lucros consistentes superiores a 99% do ótimo absoluto com tempos de execução desprezíveis.

Em última análise, a hibridização do Método Guloso com o GRASP atende aos requisitos de decisão ágeis, oferecendo um *speedup* massivo em relação aos algoritmos tradicionais sem comprometer a integridade da solução. Para investigações futuras, recomenda-se a exploração de mecanismos de diversificação de população e o uso de métricas de similaridade para o refinamento de metaheurísticas mais complexas, visando eliminar os *gaps* de otimalidade em cenários de missão crítica. Além disso, um estudo mais aprofundado sobre a geração das instâncias de teste e análise dos cenários atuais é necessária.

Referências

- ARINA, A.; RABADI, G. Local search versus path relinking in metaheuristics: Redesigning meta-raps with application to the multidimensional knapsack problem. *Applied Soft Computing*, v. 46, p. 317–327, 2016.
- CACCHIANI, V. et al. Knapsack problems — an overview of recent advances. part ii: Multiple, multidimensional, and quadratic knapsack problems. *Computers Operations Research*, v. 143, p. 105693, 2022.
- FERNANDO, D. Q.; BALLERA, M. A. Balancing efficiency and accuracy: A hybrid approach to solving the 0/1 knapsack problem. In: *Proceedings of the 7th Artificial Intelligence and Cloud Computing Conference (AICCC 2024)*. Tokyo: ACM, 2024. p. 1–6.
- GAREY, M. R.; JOHNSON, D. S. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York: W.H. Freeman and Company, 1979.
- GLOVER, F.; KOCHENBERGER, G. A. (Ed.). *Handbook of Metaheuristics*. New York: Kluwer Academic Publishers, 2003.
- GUERROUMA, A.; AÏDER, M. A hybridized memetic algorithm for solving the multi-objective stochastic multiple knapsack problem. *Pesquisa Operacional*, Rio de Janeiro, v. 45, p. e290743, 2025.
- KAMAL, A. et al. An improved gaining sharing knowledge-based algorithm for solving resource allocation problems. *International Journal of Computational Intelligence Systems*, v. 18, n. 197, 2025.
- LE, T. A unified utility-based framework for joint scheduling and buffer management in delay tolerant networks. *Computer Communications*, v. 249, p. 108432, 2026.
- MARTELLO, S.; TOTH, P. *Knapsack Problems: Algorithms and Computer Implementations*. New York: John Wiley Sons, 1990.
- MAZUMDAR, B.; DEKA, S. K. A lightweight dtdma-assisted mac scheme for ad hoc cognitive radio iiot networks. *Electronics*, Basel, v. 14, 2025.
- PROMMAKHOT, S.; ARIMURA, M.; THOUMEUN, A. Optimal placement of electric vehicle stations using high-granularity human flow data. *Urban Science*, Basel, v. 9, n. 423, 2025.
- QIN, Z. et al. Mathematical modeling and improved memetic algorithm for the extended simplified discounted 0-1 knapsack problem with randomness. *AIMS Mathematics*, v. 10, n. 10, p. 23306–23336, 2025.
- REZOUG, A.; BADER-EL-DEN, M.; BOUGHACI, D. Hybrid genetic algorithms to solve the multidimensional knapsack problem. In: TALBI, E.-G.; NAKIB, A. (Ed.). *Bioinspired Heuristics for Optimization*. [S.l.]: Springer International Publishing, 2019. p. 235–250.
- SOUFYANE, M.; KARIM, A.; CHAKIR, L. Apwch: An adaptive profit-weight capacity heuristic for solving multi-dimensional knapsack problems. *Journal of Logistics, Informatics and Service Science*, v. 11, n. 12, p. 116–130, 2024.